

版本控制与 Git

在软件开发、文档撰写和科研项目中，文件会不断经历修改、重构、回退和多人协作。如果只依赖 `project_v1`、`project_final`、`project_final2` 这样的文件命名方式，很难准确回答“谁在什么时候修改了什么”“当前版本从哪个版本演化而来”“某次修改出现问题后如何恢复”等问题。版本控制系统（Version Control System, VCS）用于系统地记录文件历史，并提供版本比较、回退、分支开发和多人协作等能力。

版本控制系统通常分为集中式和分布式两类。Subversion（SVN）是典型的集中式版本控制系统，版本历史主要保存在中央服务器；Git 则属于分布式版本控制系统（Distributed Version Control System, DVCS），一次完整的 `git clone` 通常会在本地获得仓库的提交历史，因此大量版本查看、提交、分支和回退操作可以离线完成。Git 由 Linux Torvalds 于 2005 年开发，最初用于 Linux 内核的版本管理，目前已经成为软件开发中最常用的版本控制工具之一。

Git 的核心特点包括本地操作速度快、分支创建和合并成本低、适合分布式协作，以及通过内容哈希标识 Git 对象。传统 Git 仓库默认使用 SHA-1 作为对象标识算法，新版本 Git 已经具备向 SHA-256 迁移的相关机制，因此不应简单地把 Git 描述为“永远使用 SHA-1”。哈希机制能够帮助 Git 识别对象并检查数据是否发生意外损坏，但版本控制本身不能替代权限管理、备份和安全审计。

Git 与 GitHub

Git 和 GitHub 需要明确区分。**Git 是运行在本地计算机上的分布式版本控制系统，GitHub 是提供 Git 仓库托管和协作功能的在线平台。**即使完全不使用 GitHub，也可以在本地正常使用 Git；同样，GitHub 也只是远程仓库服务的一种选择，常见替代平台还包括 GitLab、Bitbucket 和 Gitee 等。

GitHub 在 Git 仓库托管之外增加了 Pull Request、Issues、Actions、项目管理、代码评审和权限控制等功能。在实际开发中，可以把 Git 理解为负责“版本历史和同步机制”的工具，把 GitHub 理解为负责“远程托管和团队协作”的平台。

环境搭建：Git 安装与基础配置

Linux 安装

Linux 上通常直接通过发行版的软件包管理器安装 Git。以 Ubuntu、Debian 及其衍生发行版为例，可以执行：

```
sudo apt update
sudo apt install git
```

Fedora、RHEL 等使用 `dnf` 的发行版可以执行：

```
sudo dnf install git
```

Arch Linux 及其衍生发行版可以使用：

```
sudo pacman -S git
```

安装完成后使用下面的命令检查 Git 是否可用：

```
git --version
```

如果正在使用 WSL，需要注意 Windows 与 WSL 是两个相对独立的用户环境。即使 Windows 中已经安装 Git for Windows，通常仍建议在 WSL 的 Linux 发行版内部单独安装 Git，并在 WSL 中单独配置用户名、邮箱和 SSH 密钥，这样路径、权限和开发环境更容易保持一致。

Windows 安装

Windows 可以从 Git 官方网站下载 Git for Windows 并按安装向导完成安装。安装后可以 使用 Git Bash、PowerShell 或 Windows Terminal 执行 Git 命令。对于大多数初学者，安装过程保持默认选项即可，后续再根据编辑器、终端和换行符需求进行调整。

初始化用户配置

安装完成后，应首先设置提交者姓名和邮箱。它们会写入每一次新提交的元数据中，因此应该使用长期稳定、能够识别自己的信息：

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

如果主要使用 GitHub，可以使用 GitHub 账户绑定的邮箱，也可以使用 GitHub 提供的隐私邮箱。`--global` 表示该配置作为当前用户的默认配置应用于所有仓库；如果某个仓库需要使用不同身份，可以进入该仓库后去掉 `--global` 单独配置。

为了统一新仓库的初始分支名称，可以显式设置：

```
git config --global init.defaultBranch main
```

当前不同 Git 版本和用户配置下，`git init` 创建的初始分支名称可能并不完全一致，因此比起假设一定是 `master` 或一定是 `main`，显式配置更加可靠。最后可以使用以下命令检查配置：

```
git config --list
```

如果只想查看某一项，可以使用：

```
git config --global user.name
git config --global user.email
```

Linux 下连接 GitHub: SSH 与 HTTPS

本地 Git 只负责版本控制，当需要从 GitHub 克隆私有仓库或向远程仓库推送代码时，还需要完成身份认证。GitHub 常用的命令行认证方式包括 SSH 和 HTTPS。Linux 开发环境中长期使用时，SSH 通常更加方便；HTTPS 同样可以使用，但 GitHub 的 Git 操作已经不再支持直接使用账户密码认证，需要使用 Personal Access Token（PAT）、Git Credential Manager 或 GitHub CLI 等方式。

使用 SSH 认证

首先检查是否已经存在 SSH 密钥：

```
ls -al ~/.ssh
```

如果没有合适的密钥，可以生成 Ed25519 密钥：

```
ssh-keygen -t ed25519 -C "youremail@example.com"
```

按提示选择保存路径。对于普通个人电脑，直接使用默认路径 `~/.ssh/id_ed25519` 即可；是否设置密钥口令可以根据安全需求决定。随后启动 `ssh-agent` 并加载私钥：

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

查看公钥内容：

```
cat ~/.ssh/id_ed25519.pub
```

将输出的整行公钥复制到 GitHub 的 [Settings → SSH and GPG keys → New SSH key](#) 中。私钥文件 `~/.ssh/id_ed25519` 必须保留在本机并妥善保管，上传到 GitHub 的只能是带有 `.pub` 后缀的公钥。

配置完成后可以测试连接：

```
ssh -T git@github.com
```

首次连接时可能需要确认 GitHub 主机指纹。如果认证成功，之后就可以使用 SSH 地址进行克隆和推送，例如：

```
git clone git@github.com:USER/REPOSITORY.git
```

使用 HTTPS 认证

HTTPS 仓库地址通常形如：

```
git clone https://github.com/USER/REPOSITORY.git
```

公开仓库的克隆不需要登录，但向 GitHub 推送或访问需要权限的仓库时需要身份认证。GitHub 已经取消 Git 命令行中的账户密码认证，因此提示输入“Password”时不能填写 GitHub 登录密码，应使用 Personal Access Token，或者配置 Git Credential Manager、GitHub CLI 等凭据管理方式。对于固定 Linux 开发机，如果没有企业代理等特殊限制，使用 SSH 可以减少重复认证配置。

Git 的核心模型与本地工作流

工作区、暂存区与本地仓库

理解 Git 最重要的是区分工作区（Working Tree）、暂存区（Staging Area / Index）和本地仓库（Repository）。工作区就是实际编辑源码和文档的目录；暂存区保存“下一次提交准备记录哪些内容”；本地仓库位于项目的 `.git` 目录中，保存提交历史、分支引用和 Git 对象等信息。一次最常见的工作过程可以概括为：在工作区修改文件，通过 `git add` 把需要记录的内容加入暂存区，再通过 `git commit` 将暂存区快照写入本地仓库。

需要特别注意，Git 并不会在执行 `git init` 后自动“持续追踪所有文件”。新文件最初处于 Untracked 状态，只有加入暂存区并提交后才会进入版本历史；已经被追踪的文件发生变化后，Git 才会记录其差异状态。

初始化仓库

对于一个已经存在的本地项目，可以进入项目根目录执行：

```
git init
```

如果希望初始化时直接将初始分支命名为 `main`，可以使用：

```
git init -b main
```

执行后当前目录会出现隐藏的 `.git` 目录。删除 `.git` 会删除该目录的 Git 版本历史和仓库配置，因此不要把它当成普通缓存目录处理。

查看状态与差异

`git status` 是日常使用频率最高的命令之一，它会显示当前分支、未跟踪文件、已修改文件以及已经进入暂存区的内容：

```
git status
```

在提交之前，仅看状态通常还不够，还应该查看实际修改了什么。下面两个命令分别查看“工作区相对暂存区的变化”和“暂存区相对最近一次提交的变化”：

```
git diff  
git diff --staged
```

养成在 `git add` 和 `git commit` 前后查看 `status` 与 `diff` 的习惯，可以显著减少误提交文件、调试输出和无关修改的情况。

暂存与提交

将单个文件加入暂存区：

```
git add <file>
```

将当前目录及其子目录中的相关新增、修改和删除加入暂存区：

```
git add .
```

随后创建提交：

```
git commit -m "说明本次修改"
```

一次提交最好对应一个相对独立、能够说明目的的改动，例如“修复串口解析越界”或“增加机器人配置文件”，而不是把多个互不相关的工作堆在同一次提交中。Git 的价值很大程度上来自清晰的历史，因此提交频率通常应该高于“整个功能完全做完以后再提交一次”。

查看提交历史

查看完整提交历史：

```
git log
```

常用的简洁形式为：

```
git log --oneline --graph --decorate --all
```

其中 `--oneline` 将每次提交压缩为一行，`--graph` 显示分支拓扑，`--decorate` 显示分支和标签名称，`--all` 显示所有本地可见引用。提交前面的十六进制字符串是 Commit ID 的缩写，它来自 Git 对象的哈希标识，可以用于 `git show`、`git revert`、`git reset` 等操作。

查看某次提交的具体内容可以使用：

```
git show <commit-id>
```

.gitignore：避免提交无关文件

实际项目中通常存在编译产物、缓存、日志、IDE 配置、数据集和模型权重等不适合进入普通 Git 历史的文件，可以在项目根目录创建 `.gitignore` 进行排除。例如：

```
# Python
__pycache__/
*.pyc
.venv/

# C/C++ build
build/
cmake-build-*/

# IDE
.vscode/
.idea/

# Logs and temporary files
*.log
*.tmp

# Large generated artifacts
*.bag
*.pt
*.pth
```

`.gitignore` 只会阻止尚未被 Git 跟踪的文件进入版本控制。如果某个文件已经提交过，后来再把它写入 `.gitignore`，Git 仍然会继续追踪它。此时可以在保留本地文件的同时取消 Git 跟踪：

```
git rm --cached <file>
```

对于必须版本化的大型二进制文件，可以进一步了解 Git LFS；对于可重新生成的构建产物、数据缓存、训练日志和模型中间文件，通常更适合直接忽略。普通 Git 仓库并不适合长期保存频繁变化的大型二进制数据。

Git 分支与合并

分支的本质与用途

Git 分支本质上是指向某次提交的可移动引用。创建分支不会复制整个项目目录，因此操作非常轻量。开发新功能、修复 Bug 或进行实验时，可以从稳定分支创建独立分支，在完成并验证后再合并回主线。对于个人项目也推荐使用分支，因为它能把“稳定状态”和“正在尝试的修改”分离开。

常见团队会把 `main` 作为稳定主分支，再为具体任务创建 `feature/...`、`fix/...` 等短期分支。Git Flow 是一种历史上较常见的分支模型，但并非所有团队都需要长期维护 `develop` 等多层分支；小型项目和持续集成项目往往使用更简单的短生命周期功能分支工作流。

创建、切换和删除分支

查看本地分支：

```
git branch
```

创建并切换到新分支：

```
git switch -c feature/example
```

切换已有分支：

```
git switch main
```

旧版教程中常见的 `git checkout` 仍然可以完成分支切换，但较新的 Git 将“切换分支”和“恢复文件”分别拆分为 `git switch` 与 `git restore`，语义更清楚，因此本文优先采用新命令。

当某个已经合并的本地分支不再需要时，可以删除：

```
git branch -d feature/example
```

如果分支仍包含未合并提交，`-d` 会拒绝删除。`git branch -D` 可以强制删除，但执行前应先确认这些提交确实不再需要。

合并分支

假设功能分支 `feature/example` 已经完成，希望合并到 `main`，应先切换到接收修改的目标分支，再执行合并：

```
git switch main  
git merge feature/example
```

如果 `main` 从分支创建后没有产生新的分叉提交，Git 可能执行 Fast-forward，仅把分支指针向前移动；如果两个分支都各自产生了提交，则通常需要进行三方合并，并可能创建 Merge Commit。

合并冲突

当两个分支修改了同一文件中无法自动兼容的区域时，Git 会停止合并并标记冲突。此时首先运行：

```
git status
```

冲突文件中通常会出现：

```
<<<<<<< HEAD
当前分支内容
=====
待合并分支内容
>>>>>> feature/example
```

应根据实际逻辑编辑文件，保留最终正确内容并删除这些冲突标记，然后将解决后的文件重新加入暂存区：

```
git add <conflicted-file>
```

所有冲突处理完成后，根据 Git 当前提示执行 `git commit` 或 `git merge --continue` 完成合并。如果发现这次合并方向错误且尚未完成，可以使用 `git merge --abort` 尝试回到合并开始前的状态。

远程仓库与 GitHub 协作

克隆已有仓库

加入已有项目时，通常直接克隆远程仓库：

```
git clone git@github.com:USER/REPOSITORY.git
```

也可以使用 HTTPS 地址。`git clone` 会创建本地仓库、下载远程历史，并通常将该远程仓库命名为 `origin`。进入仓库后可以查看远程信息：

```
git remote -v
```

将本地项目发布到 GitHub

对于已经在本地完成 `git init` 和至少一次提交的项目，建议在 GitHub 上创建空仓库，不要预先添加 README、LICENSE 或 `.gitignore`，这样可以避免本地历史和远程历史从两个独立根提交开始。随后在本地添加远程地址：

```
git remote add origin git@github.com:USER/REPOSITORY.git
git remote -v
```

确认当前主分支名称后首次推送，例如：

```
git push -u origin main
```

`-u` 会为本地 `main` 设置上游分支。以后位于该分支时通常可以直接执行 `git push` 和 `git pull`。

如果远程仓库已经预先创建了 README 等提交，而本地项目也拥有独立提交历史，对于初学者更稳妥的做法通常是先克隆远程仓库，再把本地项目文件复制或迁移进去后正常提交。直接强行合并两个无共同祖先的历史需要理解 `--allow-unrelated-histories` 等机制，不建议在不清楚提交关系时机械执行命令。

fetch、pull 与 push

`git fetch` 从远程获取新的提交和引用，但不会直接修改当前工作分支：

```
git fetch origin
```

获取后可以查看远程跟踪分支，例如：

```
git log --oneline main..origin/main
git diff main origin/main
```

`git pull` 会先执行获取操作，然后将远程上游分支整合到当前分支。这里不应简单记成永远等于 `git fetch + git merge`，因为整合阶段可能根据命令参数和配置采用 merge、rebase 或仅允许 fast-forward 等策略。对于初学阶段，可以把它理解为“获取远程更新并尝试整合到当前分支”，出现分叉时先查看 Git 提示，再决定采用哪种策略。

```
git pull
```

`git push` 用于把本地已经产生的提交发送到远程：

```
git push
```

首次推送新分支时常用：

```
git push -u origin feature/example
```

日常协作中应避免无必要的强制推送，因为它会重写远程分支历史。如果确实理解后果并需要更新自己独占的远程分支，`git push --force-with-lease` 通常比直接 `--force` 更安全，因为它会检查远程分支是否出现了自己尚未看到的新提交。

一个常见的团队工作流程

实际开发中可以采用下面的简单流程：先同步主分支，在主分支基础上创建任务分支，完成一组相关修改后提交，再把任务分支推送到 GitHub，通过 Pull Request 进行审查和合并。对应命令大致如下：

```
git switch main
git pull

git switch -c feature/example
# 修改文件
git status
git diff
git add .
git diff --staged
git commit -m "feat: add example feature"

git push -u origin feature/example
```

这种流程的关键并不在于记住固定命令顺序，而在于始终知道“当前在哪个分支”“哪些修改还在工作区”“哪些修改已经暂存”“哪些修改已经提交”“哪些提交只存在本地”“哪些提交已经发布到远程”。遇到问题时优先查看 `git status`、`git log --graph` 和 `git remote -v`，比直接尝试大量回滚命令更加可靠。

撤销、恢复与版本回退

Git 的撤销操作需要根据错误发生在哪一层来选择命令。对初学者而言，首先区分“工作区修改”“已经暂存”“已经提交但未发布”和“已经推送到共享远程”四种情况，可以避免大量误操作。

取消暂存

如果文件已经执行 `git add`，但希望把它从暂存区移回工作区，同时保留文件内容，推荐使用：

```
git restore --staged <file>
```

这与旧教程中常见的 `git reset HEAD <file>` 用途相近，但 `restore --staged` 的语义更加直接。

放弃工作区修改

如果确定某个已跟踪文件的当前修改完全不需要，可以恢复为暂存区中的版本：

```
git restore <file>
```

这会覆盖该文件当前未提交的修改，因此执行前应先通过 `git diff <file>` 确认。对于尚未进入任何提交、暂存或 stash 的内容，Git 未必能够帮助恢复，所以不要把 `restore` 当作可以随意反悔的无风险操作。

修改最近一次提交

如果最近一次提交的说明写错，或者漏加了少量文件，并且该提交尚未发布到共享分支，可以使用：

```
git add <forgotten-file>
git commit --amend
```

`--amend` 会创建一个新的提交替换原来的最后一次提交，因此 Commit ID 会变化。如果旧提交已经被其他人拉取，再执行 `amend` 并强制推送会造成历史分叉，所以公共历史上应谨慎使用。

reset 与 revert

`git reset` 主要用于移动当前分支指针，并根据模式决定是否同时修改暂存区和工作区。最常见的三种模式是：

```
git reset --soft <commit-id>
git reset --mixed <commit-id>
git reset --hard <commit-id>
```

`--soft` 只移动提交位置，修改仍保留在暂存区；`--mixed` 还会重置暂存区，但通常保留工作区文件；`--hard` 会同时重置分支、暂存区和工作区，因此风险最高。需要纠正一个常见说法：`reset --hard` 之后的提交并不一定会立即从磁盘上永久消失，它们在一段时间内可能仍能通过 `reflog` 找回，但未提交的工作区修改可能无法恢复，所以不应依赖这种恢复可能性。

对于已经发布到共享分支的错误提交，更常用 `git revert`：

```
git revert <commit-id>
```

`revert` 会生成一个新的提交来反向抵消目标提交的修改，从而保留原有历史。公共分支上通常优先采用这种方式，因为其他协作者可以通过正常拉取获得撤销结果，而无需共同处理被改写的提交历史。

场景	推荐操作	是否改写已有提交历史
文件已暂存但不想提交	<code>git restore --staged <file></code>	否
放弃未提交的工作区修改	<code>git restore <file></code>	否
修正尚未发布的最后一次提交	<code>git commit --amend</code>	是
回退本地私有分支	<code>git reset</code>	是
撤销已经发布的公共提交	<code>git revert <commit></code>	否

使用 `reflog` 处理误操作

如果误执行了 `reset`、错误切换了分支或一时找不到刚才的提交，可以先查看本地引用操作记录：

```
git reflog
```

`reflog` 经常能够找到刚刚丢失的提交 ID，再通过创建分支、`reset` 或其他方式恢复。它是处理本地 Git 误操作的重要工具，但 `reflog` 也会过期，因此仍然不能代替正常提交和远程备份。

常用辅助功能

临时保存工作：`git stash`

当工作区还有未完成修改，但需要临时切换分支处理其他任务时，可以使用 `stash`：

```
git stash push -m "temporary work"
git stash list
git stash pop
```

`stash` 适合短期保存尚不适合提交的修改。对于已经形成明确阶段成果的工作，正常创建提交通常更加可靠，也更容易追踪。

标签：`git tag`

标签常用于标记发布版本或重要里程碑，例如：

```
git tag -a v1.0.0 -m "release v1.0.0"
git push origin v1.0.0
```

分支会随着新提交移动，而标签通常固定指向某个特定提交，因此适合标记稳定发布版本。

Rebase 的基本认识

`git rebase` 可以把一系列提交重新应用到新的基准提交上，从而形成更线性的历史，但它会改变被重新应用提交的 Commit ID。对尚未发布的个人功能分支进行 rebase 很常见，对已经被多人共享的提交进行 rebase 则需要十分谨慎。初学阶段应先熟练掌握 `commit`、`branch`、`merge`、`fetch`、`pull`、`push` 和基本撤销，再进一步学习交互式

rebase。

常用命令速查

分类	命令	作用
安装检查	<code>git --version</code>	查看 Git 版本
用户配置	<code>git config --global user.name "Name"</code>	设置全局提交用户名
用户配置	<code>git config --global user.email "Email"</code>	设置全局提交邮箱
用户配置	<code>git config --global init.defaultBranch main</code>	设置新仓库默认初始分支名
创建仓库	<code>git init</code>	在当前目录初始化仓库
创建仓库	<code>git clone <url></code>	克隆远程仓库
状态检查	<code>git status</code>	查看工作区和暂存区状态
差异检查	<code>git diff</code>	查看尚未暂存的修改
差异检查	<code>git diff --staged</code>	查看已经暂存的修改
暂存	<code>git add <file></code>	暂存指定文件
暂存	<code>git add .</code>	暂存当前目录相关修改
提交	<code>git commit -m "message"</code>	创建本地提交
历史	<code>git log --oneline --graph --decorate --all</code>	查看简洁分支历史
分支	<code>git switch -c <branch></code>	创建并切换到新分支
分支	<code>git switch <branch></code>	切换分支
分支	<code>git merge <branch></code>	将指定分支合并到当前分支
远程	<code>git remote -v</code>	查看远程仓库地址
远程	<code>git fetch</code>	获取远程更新但不直接整合到当前分支

分类	命令	作用
远程	<code>git pull</code>	获取并整合上游分支更新
远程	<code>git push</code>	推送本地提交到远程
撤销暂存	<code>git restore --staged <file></code>	保留修改并移出暂存区
放弃修改	<code>git restore <file></code>	放弃已跟踪文件的工作区修改
修改提交	<code>git commit --amend</code>	重写最近一次提交
公共撤销	<code>git revert <commit></code>	用新提交撤销历史提交
本地回退	<code>git reset <mode> <commit></code>	移动当前分支并按模式处理暂存区/工作区
恢复辅助	<code>git reflog</code>	查看本地引用移动历史
临时保存	<code>git stash</code>	临时保存未完成修改

Commit Message 的基本规范

Git 本身并不强制提交信息采用固定格式，但清晰的 Commit Message 会直接影响后续调试、代码评审和版本追踪。一个简单原则是：主题行准确说明“这次提交完成了什么”，正文在有必要时再解释“为什么这样改”以及存在什么限制。

很多项目会采用 Conventional Commits 风格的前缀，这是一种常见约定，但并非 Git 自身要求：

```
<type>: <subject>

[optional body]

[optional footer]
```

常见类型包括 `feat`（新增功能）、`fix`（修复问题）、`docs`（文档修改）、`refactor`（重构）、`test`（测试相关）、`style`（仅代码格式变化）和 `chore`（构建或辅助任务）。例如：

```
feat: add motor state timeout detection
```

```
Detect stale motor feedback and report communication faults.
```

提交信息应尽量描述实际改动，避免大量使用 `update`、`change something`、`fix bug` 这类无法从历史中判断具体内容的表述。

总结

Git 的基础学习不需要一开始记住大量高级命令，首先应建立清晰的状态模型：文件在工作区被修改，通过 `git add` 进入暂存区，通过 `git commit` 形成只存在于本地的提交，再通过 `git push` 发布到远程；`git fetch` 用于获取远程信息，`git pull` 在获取后进一步整合上游修改。分支负责隔离不同开发任务，`merge` 或后续学习的 `rebase` 负责整合历史，`restore`、`reset`、`revert` 则分别处理不同阶段的撤销需求。

日常使用中最值得形成习惯的几个命令是 `git status`、`git diff`、`git log --graph` 和 `git remote -v`。遇到 Git 问题时先确定当前分支、工作区状态、暂存区状态、本地提交历史和远程跟踪关系，再选择操作，通常比直接复制一条“强制修复”命令安全得多。